

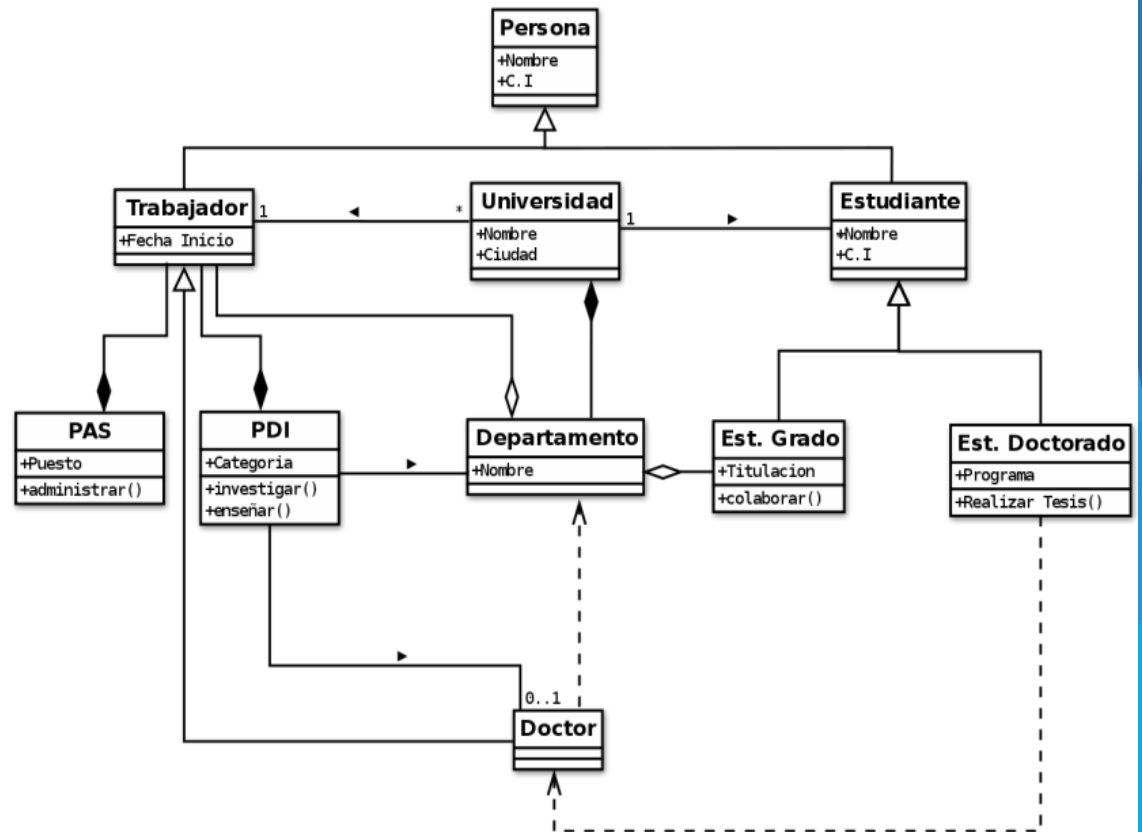
UD5.UML

UD5.1. Diagramas de clases

INTRODUCCIÓN

Los diagramas de clases son los componentes básicos del UML. Los diversos componentes en un diagrama de clases pueden representar las clases que se programarán en realidad, los objetos principales o la interacción entre clases y objetos.

Diagrama de Clases



ELABORACIÓN DE DIAGRAMAS DE CLASES

Un diagrama de clases es un tipo de diagrama de estructuras(estático) que describe la estructura de un sistema mostrando sus clases y las asociaciones entre ellas.

Sirve para visualizar las relaciones entre las clases que componen el sistema.

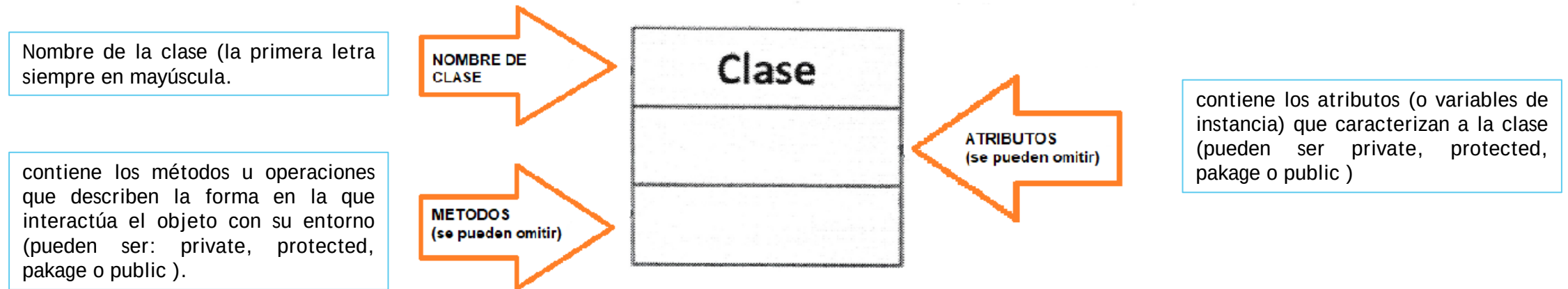
Esta compuesto por:

- ▶ **Clases:** atributos, métodos y visibilidad.
- ▶ **Relaciones:** asociación, herencia, agregación, composición, realización y dependencia.

CLASES

Como hemos visto, las **clases** son la unidad básica que encapsula toda la información de un objeto. Así, recordamos que **un objeto es una instancia de una clase**. Las clases nos permiten modelizar el entorno de nuestro sistema.

Para representar una clase, utilizaremos la estructura:



Por defecto la visibilidad de los atributos es **private** y la de los metodos **public**.

CLASES

Clases abstractas y clase concretas.

Imaginemos un ejemplo de clase que sea Animal. Como tal, no tiene características definitorias. Para poder imaginarnos uno, necesitamos concretar más.

En este ejemplo:

- ▶ Las clases que poseen instancias, por ejemplo, las clases Caballo y Perro, son llamadas clases concretas.
- ▶ Las clases que no poseen directamente instancias, como la clase Animal, se denominan clases abstractas. En efecto, si bien en el mundo real existen caballos, perros, etc., el concepto de animal propiamente dicho continúa siendo abstracto. No basta para definir completamente un animal.

La finalidad de las clases abstractas es poseer subclases concretas y sirven para factorizar atributos y métodos comunes a las subclases.

ATRIBUTOS

Un **atributo** representa alguna propiedad de la clase que se encuentra en todas las instancias de la clase. Se pueden representar por su nombre, su tipo e incluso mostrando su valor por defecto. (Ejemplos: nombre, salario, edad, etc.)

Los tipos básicos de atributos en UML son: Integer, String y Boolean.

También se pueden indicar los tipos de cualquier lenguaje de programación.

Al crear el atributo se indicará su visibilidad:

- ▶ **public (+)**: el atributo será visible tanto dentro como fuera de la clase.
- ▶ **private (-)**: el atributo solo será accesible desde dentro de la clase (solo los métodos de la clase accederán a él).
- ▶ **protected (#)**: el atributo no será accesible desde fuera de la clase, pero si por los métodos de la clase y por las subclases.
- ▶ **package (~)**: el atributo empaquetado es visible en las clases del mismo paquete.

MÉTODOS

Un **método** (operación) es la implementación de un servicio de la clase que muestra un comportamiento común a todos los objetos.

Se escribe en minúscula si consta de una sola palabra y, si son dos palabras, la segunda empieza por mayúscula. (camelCase)



Al crear el atributo se indicará su visibilidad:

- ▶ **public (+)**: el método es accesible tanto dentro como fuera de la clase.
- ▶ **private (-)**: solo los métodos de la clase accederán a él.
- ▶ **protected (#)**: el método puede ser accedido por los métodos de la clase y de las subclases.
- ▶ **package (~)**: el método empaquetado es visible en las clases del mismo paquete.

RELACIONES

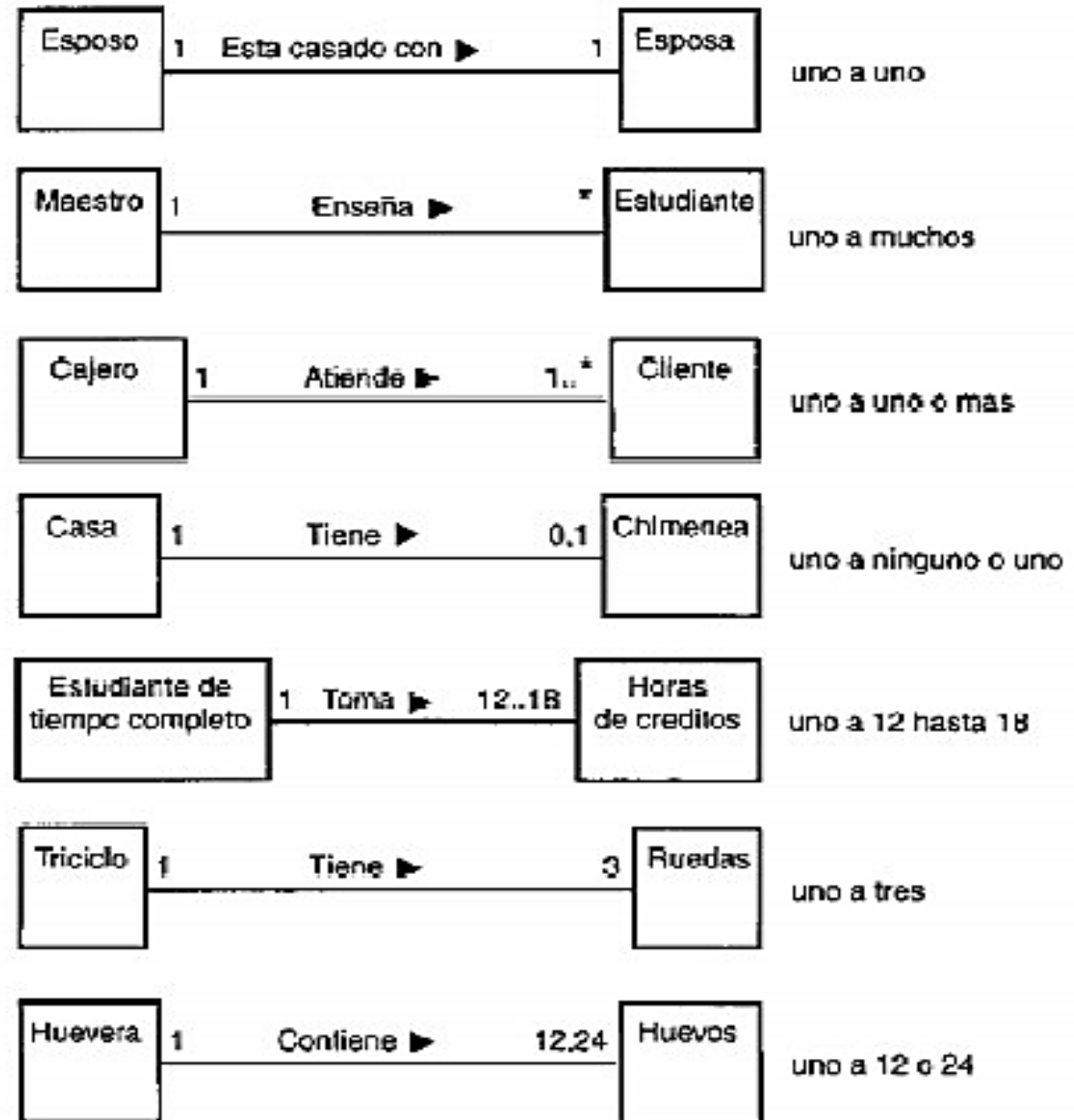
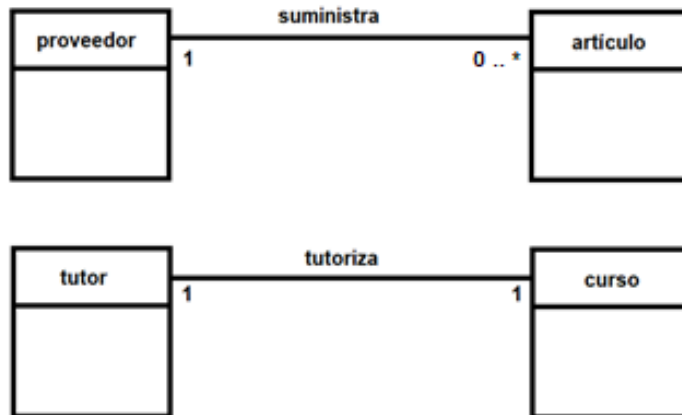
En UML, los vínculos entre los objetos (clases) se describen mediante asociaciones o **relaciones**, de igual modo que los objetos se describen mediante clases.

Las asociaciones tienen un **nombre** y una **cardinalidad** llamada **multiplicidad** que representa el número de instancias de una clase que se relacionan con las instancias de otra clase (similar a la cardinalidad utilizada en los modelos Entidad/Relación). La multiplicidad situada en un extremo de una asociación indica a cuántas instancias de la clase situada en ese mismo extremo está vinculada una instancia de la clase situada en el extremo opuesto. Para expresar las multiplicidades mínimas y máximas se utiliza la siguiente notación:

1	Uno y solo uno
0..1	Cero o uno
M..N	De M a N (enteros naturales)
*	Muchos
0..*	De cero a muchos
1..*	De uno a muchos

RELACIONES

Ejemplos:



TIPOS DE RELACIONES

En UML, para reflejar los tipos de relaciones entre los objetos se utilizan diferentes símbolos:

RELACIÓN	REPRESENTACIÓN
Asociación:	
Herencia	
Composición	
Agregación	
Realización	
Dependencia	

ASOCIACIÓN

La asociación es el tipo de relación más genérico. Es una relación entre clases que puede ser unidireccional o bidireccional, dependiendo de si ambas clases conocen la existencia la una de la otra o no.

- ▶ Cuando una asociación **bidireccional** se convierte a Java, cada clase contendrá uno o un set de objetos de la otra clase. 
- ▶ Cuando una asociación **unidireccional** se convierte a Java, la clase destino no sabrá de la existencia de la otra clase. Y la clase origen tendrá uno o un set de objetos de la clase destino. 

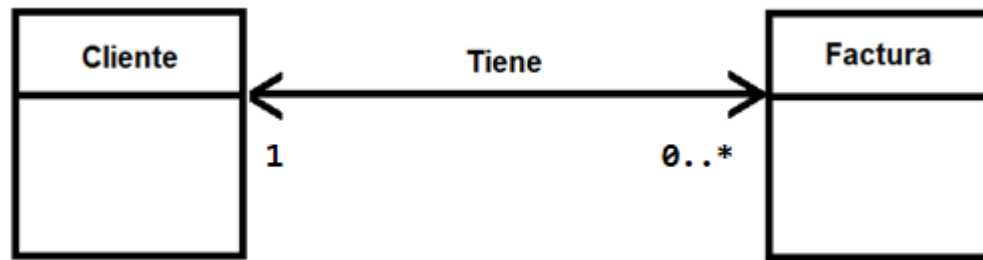
Cada clase juega un rol, que se indica en la parte superior o inferior de la línea que conecta dichas clases.

Los vínculos se representan con una línea continua a la que se le pone el nombre de la relación.

Una clase puede asociarse consigo misma creando una **asociación reflexiva**. Estas asociaciones unen entre si instancias de una misma clase.

ASOCIACIÓN

Ejemplo 1: Este diagrama de clases representa una relación uno a muchos entre Cliente y Factura. Un cliente puede tener una o más facturas, y cada factura pertenece a un único cliente.



```
class Cliente {
    private String nombre;
    private List<Factura> facturas;

    public Cliente(String nombre) {
        this.nombre = nombre;
        this.facturas = new ArrayList<>();
    }
}
```

```
public class Factura {
    private int numero;
    private Cliente cliente;

    public Factura(int numero, Cliente cliente) {
        this.numero = numero;
        // Se asigna al cliente obligatoriamente
        this.setCliente(cliente);
    }
}
```

ASOCIACIÓN

Ejemplo 2: Un almacén distribuye artículos en varias zonas. Sólo la clase origen Almacén conoce la existencia de la clase destino Zona.



```
public class Zona {
    private String nombre;
}
```

```
public class Almacen {
    private String nombre;
    private List<Zona> zonas;

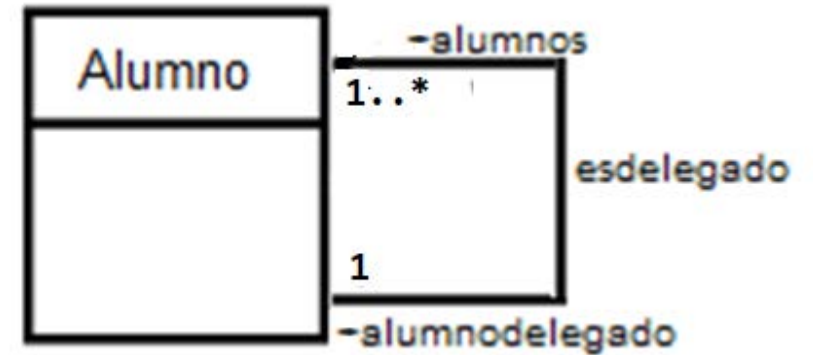
    public Almacen(String nombre, Zona zona) {
        this.nombre = nombre;
        this.zonas = new ArrayList<>();
        this.zonas.add(zona);
    }
}
```

ASOCIACIÓN

Ejemplo 3: Un alumno es delegado de varios alumnos.

Cada alumno tiene un único delegado.

- ▶ La relación **esdelegado** muestra que un solo alumno (1) puede ser delegado de uno o más alumnos (1..*).
- ▶ La relación inversa indica que un conjunto de **alumnos** (1..*) puede tener exactamente un **alumnodelegado** (1).



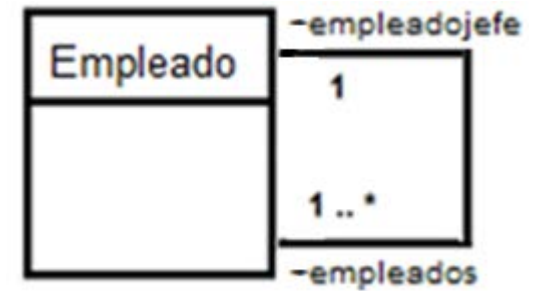
```
class Alumno {
    private String nombre;
    private Alumno delegado;
    private ArrayList<Alumno> alumnos;

    // Constructor
    public Alumno(String nombre, Alumno delegado) {
        this.nombre = nombre;
        this.alumnos = new ArrayList<>();

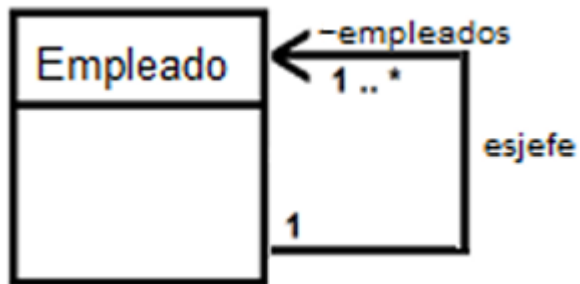
        // Si el delegado es null, se asume que este
        // alumno es su propio delegado
        if (delegado == null) {
            // Se autodesigna como su propio delegado
            this.delegado = this;
        } else {
            this.delegado = delegado;
            // Se añade automáticamente a la lista
            // del delegado
            delegado.agregarAlumno(this);
        }
    }
}
```

ASOCIACIÓN

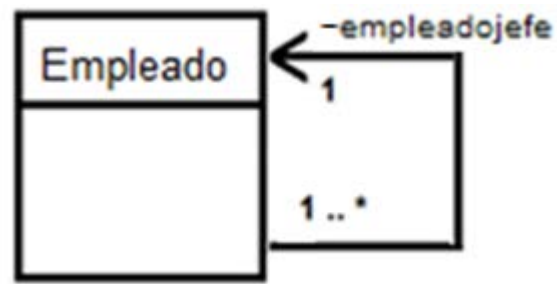
Ejemplo 4: cada empleado puede tener un jefe.
Un jefe puede supervisar a múltiples empleados.



```
class Empleado {
    private String nombre;
    // Relación de 1 a 1
    private Empleado empleadojefe;
    // Relación de 1 a muchos
    private List<Empleado> empleados;
}
```



```
class Empleado {
    private String nombre;
    // Relación de 1 a muchos
    //(el jefe conoce a sus empleados)
    private List<Empleado> empleados;
}
```



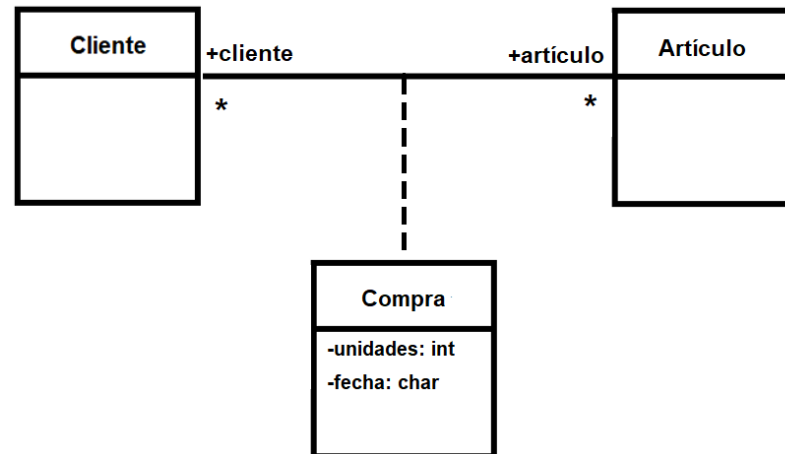
```
class Empleado {
    private String nombre;
    // Relación de 1 a 1
    //(cada empleado tiene un jefe)
    private Empleado empleadojefe;
}
```

ASOCIACIÓN

Clase asociación.

Una clase asociación en UML se utiliza para representar información adicional sobre una relación entre dos clases. Se usa cuando la relación entre objetos tiene atributos propios, que no pertenecen directamente a ninguna de las clases relacionadas. - - - - -

```
public class Cliente {  
    private String nombre;  
}
```



```
public class Articulo {  
    private String nombre;  
    private double precio;  
}
```

```
public class Compra {  
    private Cliente cliente;  
    private Articulo articulo;  
    private int unidades;  
    private Date fecha;  
}
```

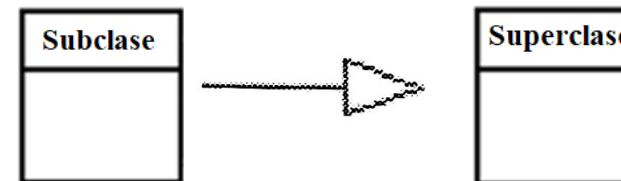

HERENCIA

Las clases con atributos y operaciones comunes se pueden organizar de forma jerárquica mediante la herencia.

La herencia es una abstracción importante para compartir similitudes entre clases, donde todos los atributos y operaciones comunes a varias clases se pueden agrupar/compartir por medio de lo que se denomina superclase, esto es, la clase más general. Las clases más refinadas se conocen como las subclases.

La **generalización** define una relación entre una clase más generalizada, y una o más versiones refinadas de ella. La generalización indica que una clase (clase secundaria o subclase) hereda los atributos y métodos de otra (clase principal o superclase).

La **especialización** es la creación de una clase más específica a partir de una clase general. Es decir, se crea una subclase que hereda los atributos y métodos de una clase base (superclase), pero con comportamientos adicionales o más específicos.

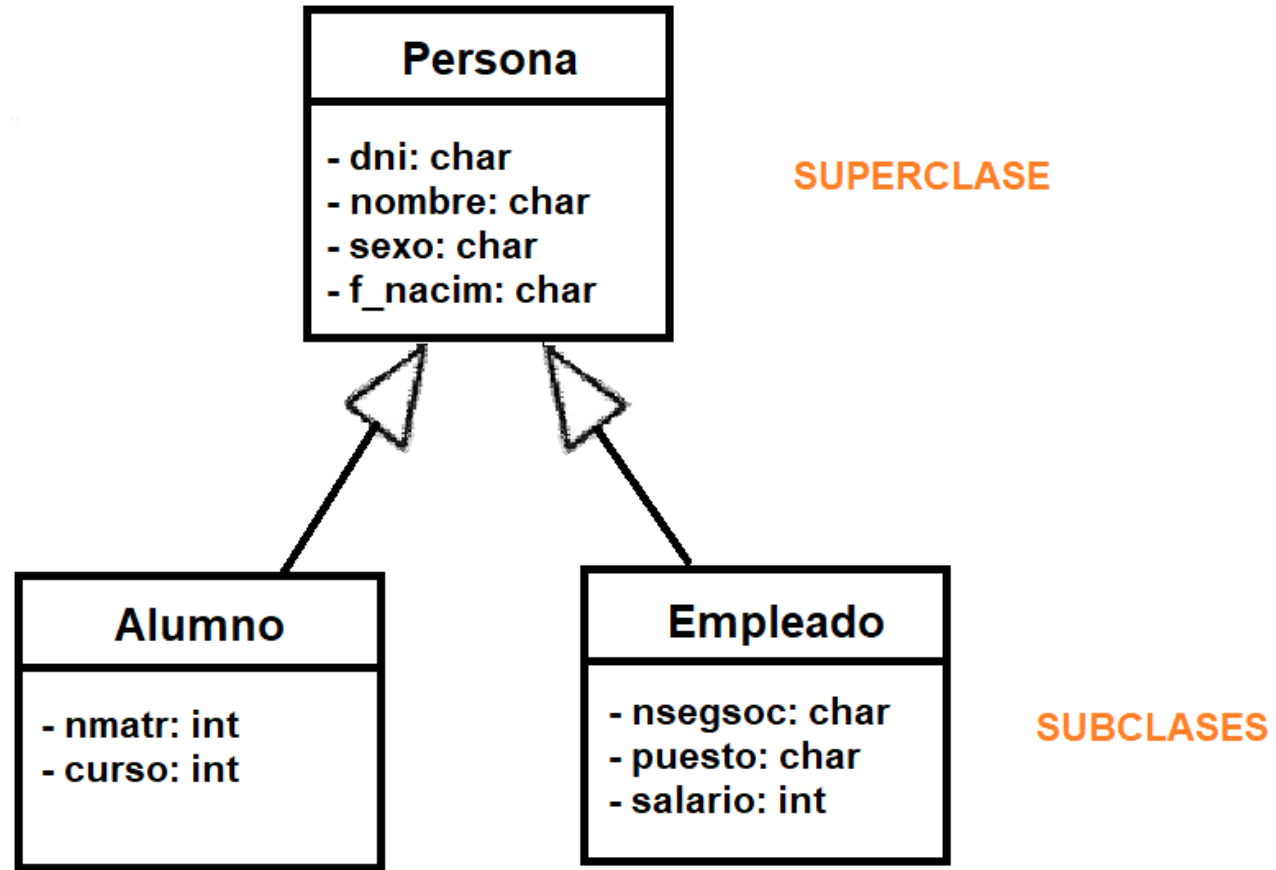


La superclase generaliza a sus subclases, y las subclases especializan a la superclase.

HERENCIA

Generalización
Especialización

```
public class Persona {  
    private char dni;  
    private char nombre;  
    private char sexo;  
    private char f_nacim;  
}
```



```
public class Alumno extends Persona {  
    private int nmatr;  
    private int curso;  
}
```

```
public class Empleado extends Persona {  
    private char nsegsoc;  
    private char puesto;  
    private int salario;  
}
```

COMPOSICIÓN Y AGREGACIÓN

En ocasiones, un objeto puede estar compuesto de otros objetos. Hay dos tipos de relaciones que pueden representar esta circunstancia. Ambas representan una relación en la que una clase contiene a otra. Sin embargo, la diferencia principal radica en cómo se maneja la existencia de los objetos dentro de la relación.

Las relaciones de **composición** y **agregación** pertenecen a la categoría de asociaciones, que describen cómo interactúan y se relacionan los objetos dentro de un sistema.

En muchos casos, un mismo sistema puede contener tanto relaciones de composición como de agregación, dependiendo de la naturaleza de los objetos y su dependencia entre sí. Esta dependencia puede ser:

- ▶ **Fuerte (composición):** en ella la existencia del objeto contenido depende totalmente del objeto contenedor.
- ▶ **Débil (agregación):** en ella la relación más flexible y, por tanto, los objetos pueden existir de manera independiente.

COMPOSICIÓN

Como hemos visto anteriormente, la **composición** es una relación **fuerte** entre dos objetos, en la cual un objeto contiene al otro. Se representa con una línea y  relleno

En este tipo cada componente **no** puede ser compartido por varios objetos compuestos. Por tanto, en el objeto compuesto como máximo la cardinalidad es 1.

Sin embargo, la cardinalidad en los objetos componentes puede ser desde 0 hasta la cantidad que necesite nuestro sistema.

En este tipo de relaciones, en el caso de que se el objeto compuesto desaparezca, desaparecen los componentes contenidos en él.

Por ejemplo, imaginemos un objeto paquete de regalo se compone de una caja y el regalo del interior. Si se destruye el regalo, desaparecen tanto la caja como el regalo contenido en ella.

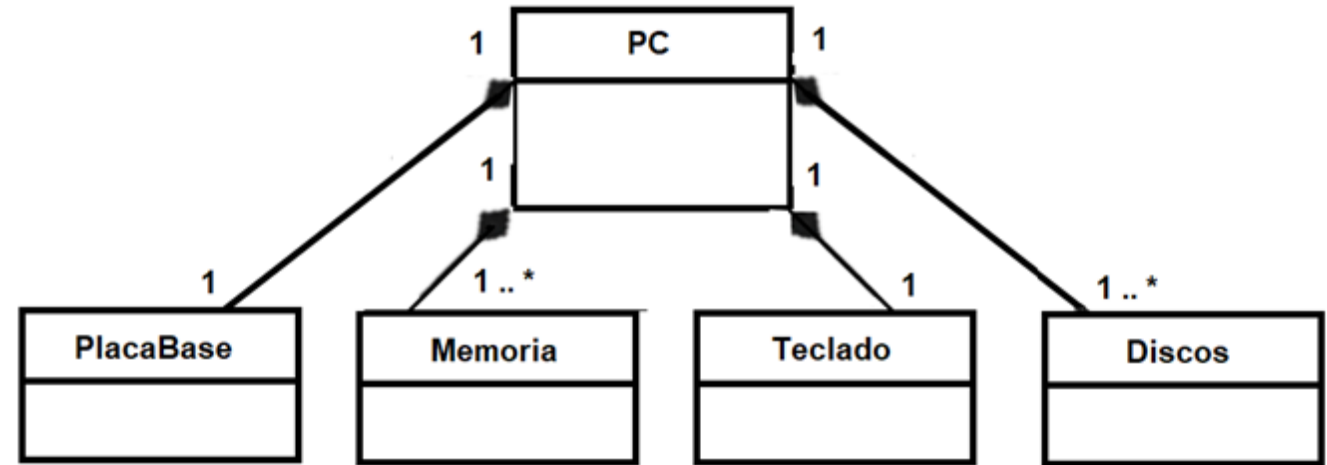
COMPOSICIÓN

```
// Clase PlacaBase (Parte obligatoria del PC)
class PlacaBase {
    private String modelo;
}

// Clase Memoria (Debe haber al menos una en el PC)
class Memoria {
    private int capacidad; // en GB
}

// Clase Teclado (Parte obligatoria del PC)
class Teclado {
    private String tipo;
}

// Clase Disco (Debe haber al menos uno en el PC)
class Disco {
    private int capacidad; // en GB
    private String tipo; // HDD o SSD
}
```



```
// Clase PC (Composición con PlacaBase, Memoria, Teclado y Disco)
class PC {
    private PlacaBase placaBase;
    private List<Memoria> memorias;
    private Teclado teclado;
    private List<Disco> discos;

    public PC(String modeloPlaca, String tipoTeclado, int capacidadMemoriaInicial,
              int capacidadDiscoInicial, String tipoDiscoInicial) {
        this.placaBase = new PlacaBase(modeloPlaca);
        this.teclado = new Teclado(tipoTeclado);
        this.memorias = new ArrayList<>();
        this.discos = new ArrayList<>();

        // Agregando al menos una memoria y un disco
        this.memorias.add(new Memoria(capacidadMemoriaInicial));
        this.discos.add(new Disco(capacidadDiscoInicial, tipoDiscoInicial));
    }
}
```

AGREGACIÓN

Como hemos visto anteriormente, la **agregación** es una relación **débil** entre dos objetos, en la cual cada componente puede ser compartido por varios objetos. Se representa con una línea y un rombo vacío 

En este tipo cada componente **si** puede ser compartido por varios objetos compuestos. Por tanto, en el objeto compuesto como máximo la cardinalidad **no** es 1.

La cardinalidad en los objetos componentes puede ser desde 0 hasta la cantidad que necesite nuestro sistema.

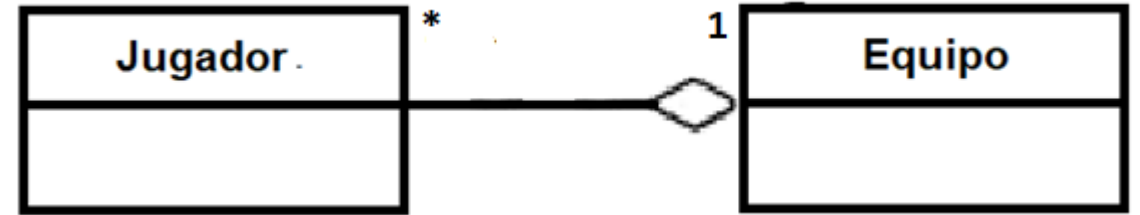
En este tipo de relaciones, en el caso de que se el objeto compuesto desaparezca, no desaparecen los componentes contenidos en él, se pueden reasignar.

Por ejemplo, imaginemos una universidad y sus profesores. Si la universidad deja de existir, los profesores pueden seguir trabajando en otro lugar de nuestro sistema.

AGREGACIÓN

En el esquema observamos que:

- ▶ Cada jugador pertenece a un único equipo a la vez.
- ▶ Cada equipo puede tener varios jugadores.
- ▶ Un jugador puede cambiar de equipo, pero siempre pertenecerá a uno en todo momento.



```
public class Jugador {
    private String nombre;
    private Equipo equipo;

    public Jugador(String nombre, Equipo equipo) {
        // El jugador siempre debe estar en un equipo
        if (equipo == null) {
            System.out.println("Falta el equipo.");
        }
        this.nombre = nombre;
        this.equipo = equipo;
        equipo.agregarJugador(this);
    }
}
```

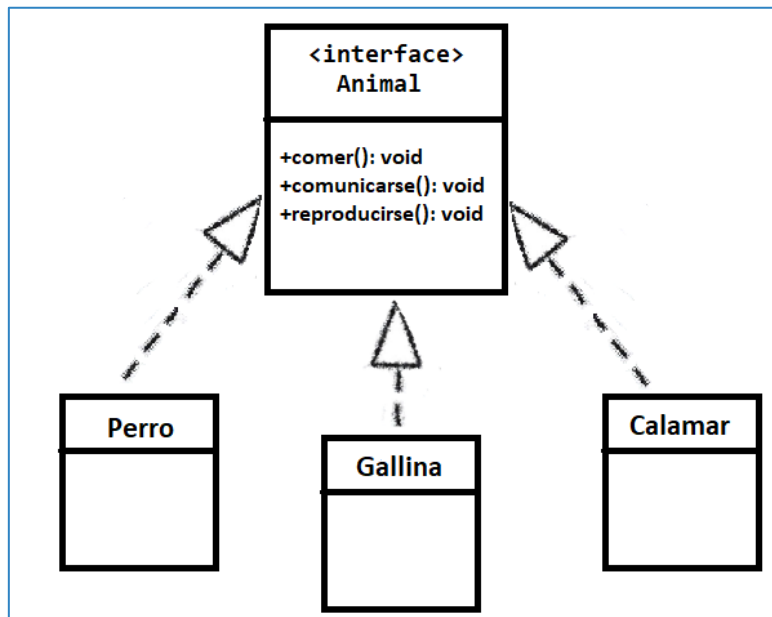
```
public class Equipo {
    private String nombre;
    private ArrayList<Jugador> jugadores;

    public Equipo(String nombre) {
        this.nombre = nombre;
        this.jugadores = new ArrayList<>();
    }
}
```

REALIZACIÓN

Es la relación de herencia existente entre una clase interfaz y la subclase que implementa esa interfaz. La interfaz nos permite especificar un conjunto de operaciones en clase que pueden ser utilizada por otras, se representa de la misma forma que una clase utilizando un rectángulo. Una interfaz es una clase, totalmente abstracta, es decir, no tiene atributos y todos sus métodos son abstractos y públicos, sin desarrollar.

Las realizaciones se representan con el símbolo: ----->



Y en Java:

<pre>public interface Animal { public void comer(); public void comunicarse (); public void reproducirse (); }</pre>	<pre>public class Perro implements Animal { public Perro (){} public void comer (){} public String comunicarse (){return "guau"} public void reproducirse (){} }</pre>
<pre>public class Gallina implements Animal { public Gallina (){} public void comer (){} public void comunicarse (){return "co-co"} public void reproducirse (){} }</pre>	<pre>public class Calamar implements Animal { public Calamar (){} public void comer (){} public void comunicarse (){return "ninguno"} public void reproducirse (){} }</pre>

REALIZACIÓN

```
class Perro implements Animal {
    @Override
    public void comer() {
        System.out.println("El perro come croquetas.");
    }

    @Override
    public void comunicarse() {
        System.out.println("¡Guau guau!");
    }

    @Override
    public void reproducirse() {
        System.out.println("El perro se tiene perritos.");
    }
}
```

```
class Gallina implements Animal {
    @Override
    public void comer() {
        System.out.println("La gallina picotea el maíz.");
    }

    @Override
    public void comunicarse() {
        System.out.println("¡Cocorocó!");
    }

    @Override
    public void reproducirse() {
        System.out.println("La gallina pone huevos.");
    }
}
```

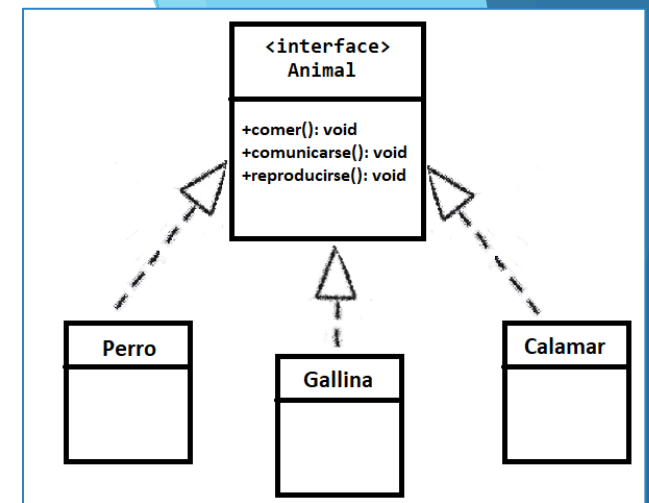
En JAVA

```
interface Animal {
    void comer();
    void comunicarse();
    void reproducirse();
}
```

```
class Calamar implements Animal {
    @Override
    public void comer() {
        System.out.println("El calamar come pequeños peces.");
    }

    @Override
    public void comunicarse() {
        System.out.println("El calamar cambia de color.");
    }

    @Override
    public void reproducirse() {
        System.out.println("El calamar libera huevos en el agua.");
    }
}
```



DEPENDENCIA



Es una relación que se establece entre dos clases cuando una clase utiliza a otra temporalmente para realizar una función específica, pero no mantiene una relación permanente con ella. Las instancias de la clase se crean y se emplean cuando se necesitan.

Las realizaciones se dependencia con el símbolo: ----->

En este ejemplo se modeliza la afirmación “el viajero necesita su equipaje para viajar”.

```
class Viajero {
    private String nombre;

    public Viajero(String nombre) {
        this.nombre = nombre;
    }

    public void viajar(String destino) {
        System.out.println(nombre + " está viajando a " + destino);

        // Dependencia: el equipaje solo se usa temporalmente en este método
        Equipaje equipaje = new Equipaje("Maleta grande");

        System.out.println("Viajando...\n");
    }
}
```

```
class Equipaje {
    private String tipo;

    public Equipaje(String tipo) {
        this.tipo = tipo;
    }
}
```